S. Rushi Keshava Reddy

192472355

CO2 – AT 2

MLA – 0302 - Reinforcement Learning
Application-Based Questions 1–10

Simple format: About the Question → How RL is Used → RL Components → Important Concept → Simple Executable Python Code.

1. Ride-Sharing Driver–Passenger Matching

About the Question
This question is about using RL to match passengers with suitable drivers, mainly to reduce waiting time and improve service quality.

How RL is Used
The system observes available drivers, passenger locations, distance and traffic. It selects a driver. A quick successful pickup gets a positive reward, while long waiting time or cancellation gets a penalty.

RL Components
Agent: Ride-sharing matching system.
Environment: Drivers, passengers, roads and traffic.
State: Driver/passenger locations, traffic and waiting time.
Action: Assign a driver to a passenger.
Reward: Positive for quick pickup; penalty for delay/cancellation.
Policy: Strategy for choosing the best match.

Main Idea
Learning from completed rides helps the system improve future matching.

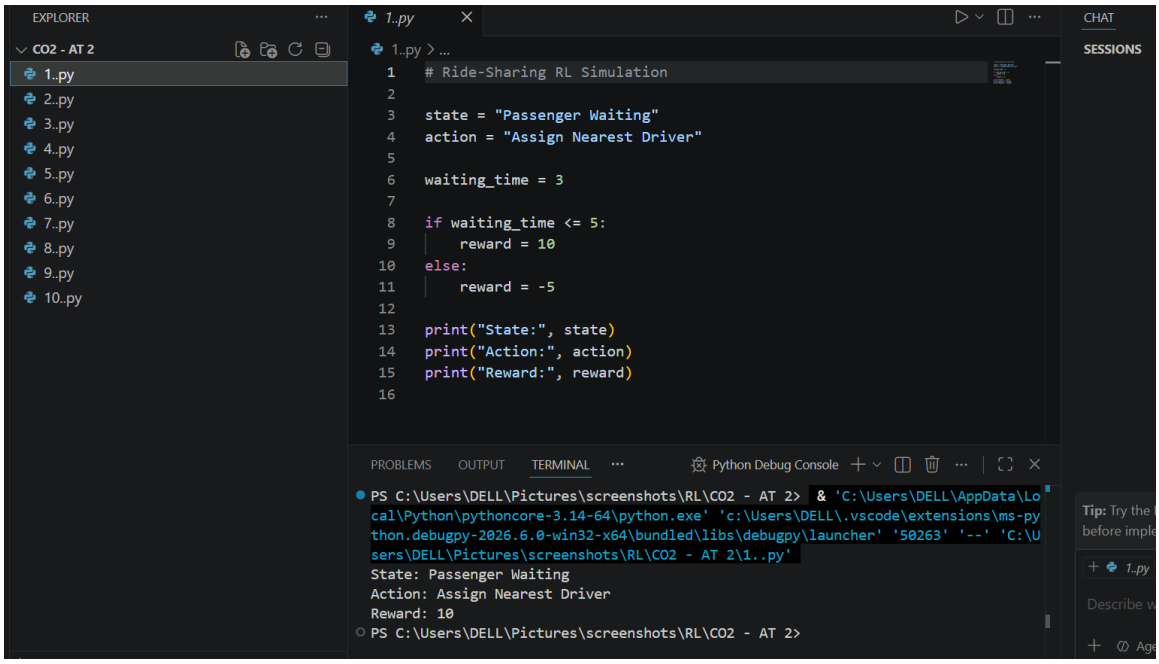
Simple Executable Python Code

```
drivers = {"Driver A": 2, "Driver B": 5, "Driver C": 3}
best_driver = min(drivers, key=drivers.get)
waiting_time = drivers[best_driver]
reward = 10 if waiting_time <= 3 else 5
print("Selected Driver:", best_driver)
```

print("Waiting Time:", waiting_time)
print("Reward:", reward)

Note: This code is intentionally simple and demonstrates the RL state–action–reward/decision idea. A complete RL system learns a policy through repeated interactions.

CODE:



2. Personalized Study Material Recommendation

About the Question

This question is about using RL in an e-learning platform to recommend suitable study materials.

How RL is Used

The agent observes student performance, interests and completed topics. It recommends familiar or new content and learns from the student's response.

RL Components

Agent: E-learning recommendation system.
Environment: Student and learning platform.

State: Performance, interests and completed topics.
Action: Recommend a video, quiz, note or topic.
Reward: Positive when the student studies, completes or performs well.
Policy: Strategy for choosing suitable material.

Exploration and Exploitation

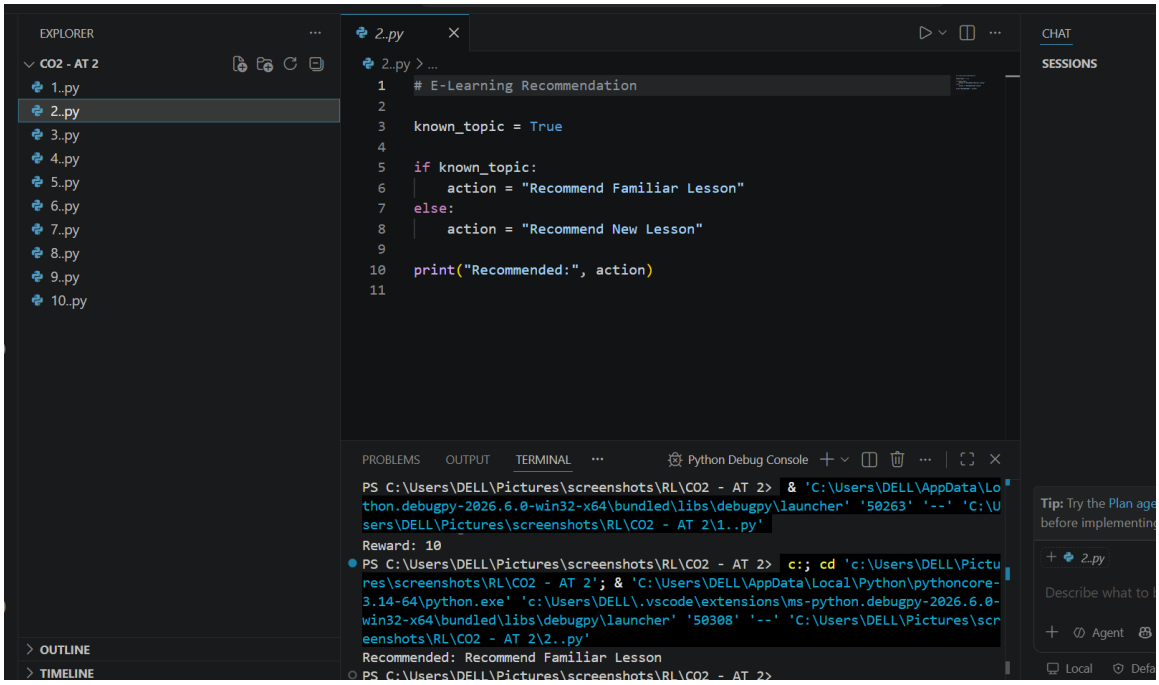
Exploration means trying new content. Exploitation means recommending familiar content already known to work. Balancing both helps discover new interests without ignoring useful known content.

Simple Executable Python Code

```
import random
familiar = "Python Basics"
new_content = "Machine Learning"
if random.random() < 0.2:
    recommendation = new_content
    strategy = "Exploration"
else:
    recommendation = familiar
    strategy = "Exploitation"
print("Strategy:", strategy)
print("Recommended:", recommendation)
```

Note: This code is intentionally simple and demonstrates the RL state–action–reward/decision idea. A complete RL system learns a policy through repeated interactions.

CODE:



3. Robotic Vacuum Cleaner

About the Question

This question is about using RL to help a robotic vacuum clean efficiently while avoiding obstacles.

How RL is Used

The robot observes its position, dirt and obstacles. It chooses movements and receives rewards for cleaning and penalties for hitting obstacles.

RL Components

Agent: Robotic vacuum.
Environment: Room, dirt, walls and obstacles.
State: Robot position, dirt and nearby obstacles.
Action: Move or clean.
Reward: Positive for cleaning; penalty for collision or wasted movement.
Policy: Strategy for choosing actions.

Policy and Value Function

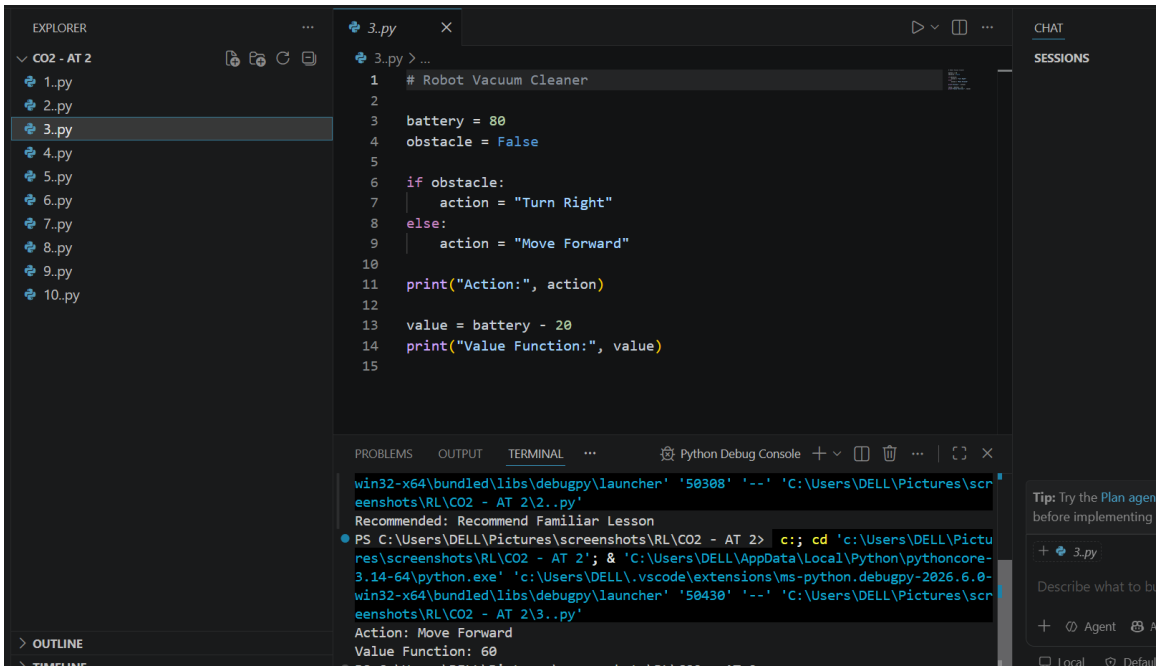
The policy tells the robot what action to take. The value function estimates how useful a state is based on future rewards. Over time, this helps the robot prefer paths that clean more efficiently.

Simple Executable Python Code

```
room = ["Clean", "Dirty", "Obstacle", "Dirty"]
for position, state in enumerate(room):
    if state == "Dirty":
        action, reward = "Clean", 10
    elif state == "Obstacle":
        action, reward = "Avoid", -5
    else:
        action, reward = "Move", 1
    print(position, state, action, reward)
```

Note: This code is intentionally simple and demonstrates the RL state–action–reward/decision idea. A complete RL system learns a policy through repeated interactions.

CODE:



4. Cryptocurrency Trading

About the Question

This question is about an RL system that chooses Buy, Sell or Hold for cryptocurrency. The main focus is reward-function design.

How RL is Used

The agent observes price, trend, holdings and available balance. It takes a trading action and later receives a reward based on the result.

RL Components

Agent: Crypto trading system.
Environment: Cryptocurrency market.
State: Price, trend, holdings and balance.
Action: Buy, Sell or Hold.
Reward: Profit as positive reward; losses, risk and costs as penalties.
Policy: Trading strategy.

Reward Design Challenges

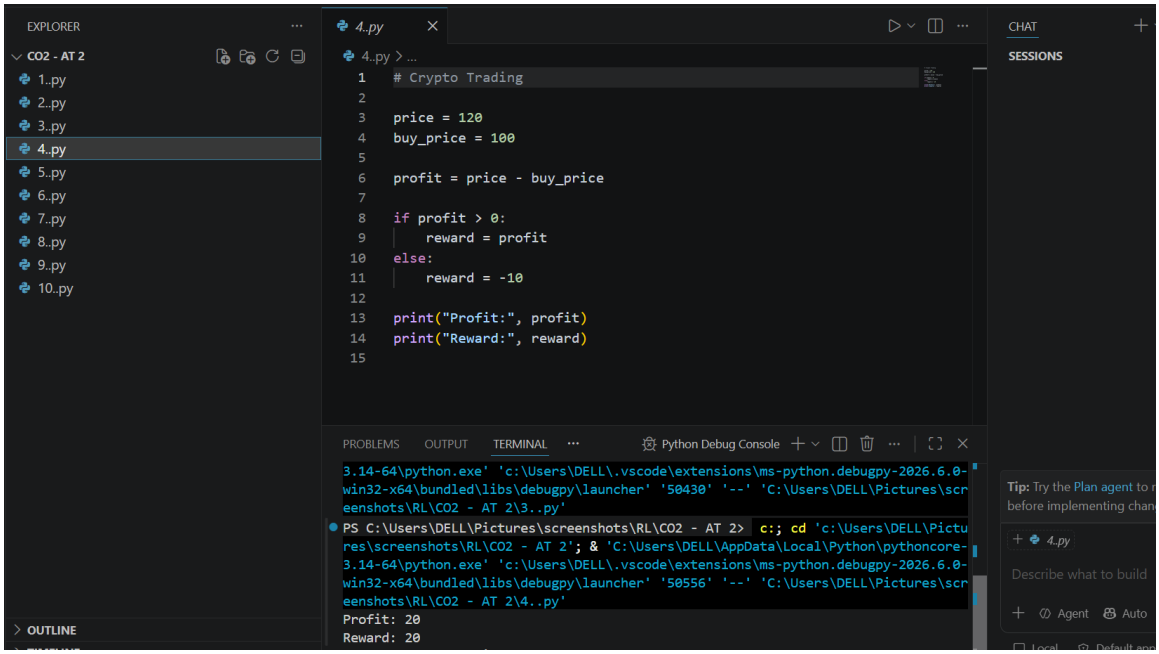
If reward only encourages short-term profit, the agent may take too much risk or trade too often. A better reward considers profit, loss, risk and transaction cost.

Simple Executable Python Code

```
buy_price = 100
later_price = 110
transaction_cost = 2
profit = later_price - buy_price - transaction_cost
reward = profit if profit > 0 else -10
print("Profit:", profit)
print("Reward:", reward)
```

Note: This code is intentionally simple and demonstrates the RL state–action–reward/decision idea. A complete RL system learns a policy through repeated interactions.

CODE:



5. Smart Grid Electricity Distribution

About the Question

This question is about using RL to distribute electricity efficiently in a smart grid.

How RL is Used

The agent observes electricity supply, demand, storage and grid load. It decides whether to supply power, store extra power or use stored power.

RL Components

Agent: Smart-grid controller.
Environment: Generators, consumers, batteries and grid.
State: Demand, supply, storage and grid load.
Action: Supply, store or use stored electricity.
Reward: Positive for meeting demand efficiently; penalty for shortage or waste.
Policy: Strategy for balancing supply and demand.

Main Idea

RL can adapt when electricity demand and generation change.

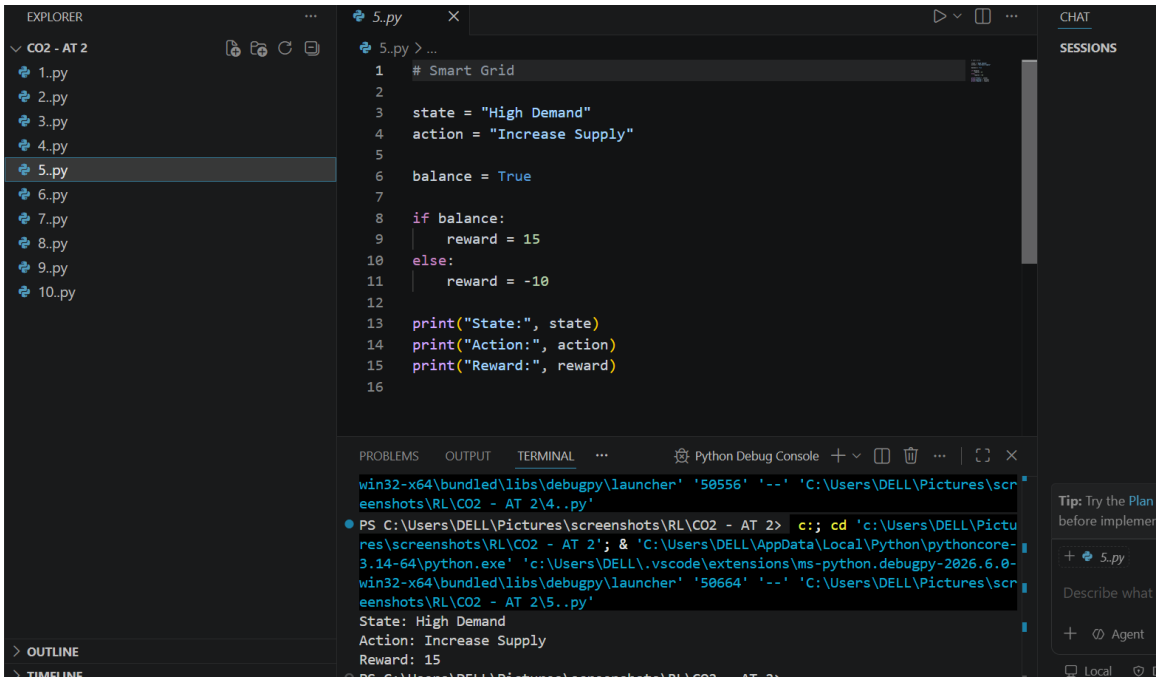
Simple Executable Python Code

```
supply = 120
demand = 90
if supply > demand:
```

```
    action = "Store Extra Power"
    reward = 10
elif supply < demand:
    action = "Use Stored Power"
    reward = 5
else:
    action = "Supply Normally"
    reward = 8
print("Action:", action)
print("Reward:", reward)
```

Note: This code is intentionally simple and demonstrates the RL state–action–reward/decision idea. A complete RL system learns a policy through repeated interactions.

CODE:



6. Healthcare Medication Dosage

About the Question

This question is about using RL in a healthcare monitoring system to support medication-dosage decisions based on patient condition.

How RL is Used

The system observes patient measurements, symptoms, previous treatment and response. It chooses among clinically approved actions and learns from outcomes. Real medical use requires doctors, validated protocols and strong safety constraints.

RL Components

Agent: Medication decision-support system.
Environment: Patient and monitoring system.
State: Patient measurements, symptoms, previous dosage and response.
Action: Maintain an approved dose or request clinical review/adjustment.
Reward: Positive for safe improvement; penalty for poor or unsafe outcomes.
Policy: Strategy for selecting an approved action.

Learning

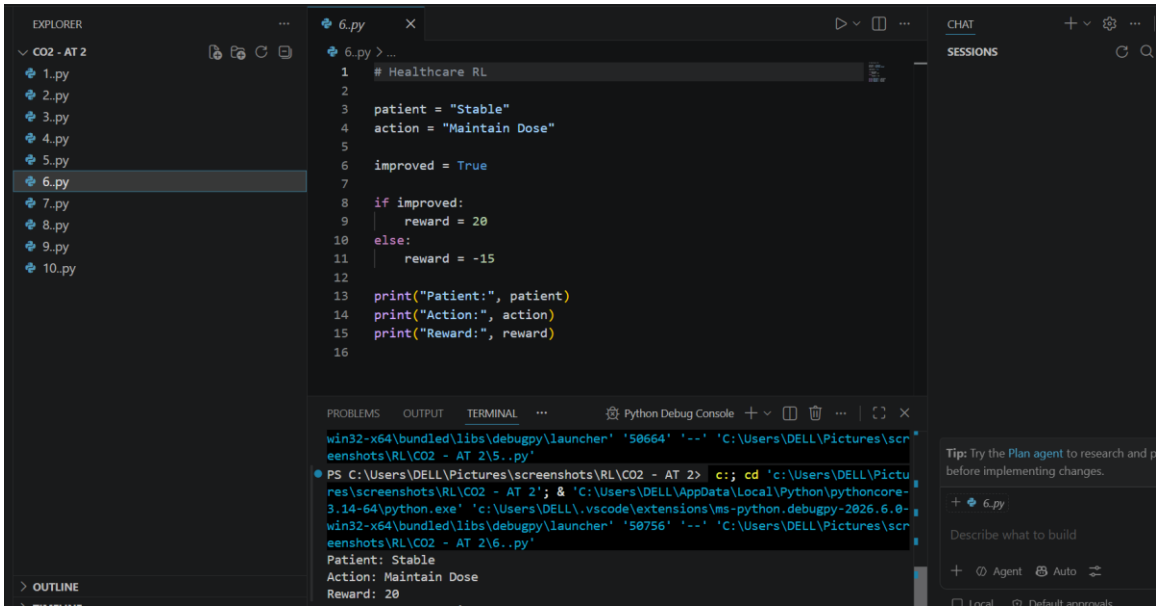
Feedback can help an RL research system associate actions with better outcomes, but safety and professional oversight are essential.

Simple Executable Python Code

```
patient_response = "Improving"
if patient_response == "Improving":
    action, reward = "Maintain Approved Dose", 10
elif patient_response == "No Change":
    action, reward = "Request Clinical Review", 5
else:
    action, reward = "Immediate Clinical Review", -5
print("Decision:", action)
print("Reward:", reward)
```

Note: This code is intentionally simple and demonstrates the RL state–action–reward/decision idea. A complete RL system learns a policy through repeated interactions.

CODE:



7. Drone Delivery Navigation

About the Question
This question is about using RL to help a delivery drone find safe and efficient routes.

How RL is Used
The drone observes location, destination, obstacles, battery and weather. It selects movements or routes. Safe short routes get positive rewards; collisions and long routes get penalties.

RL Components
Agent: Delivery drone.
Environment: Airspace, obstacles, weather and destination.
State: Location, destination, battery, obstacles and weather.
Action: Move or choose a route.
Reward: Positive for safe delivery; penalty for collision, delay or high energy use.
Policy: Strategy for navigation.

Exploration and Exploitation
Exploration tries alternative routes to discover better paths. Exploitation uses the best route already learned. A balance improves route optimization.

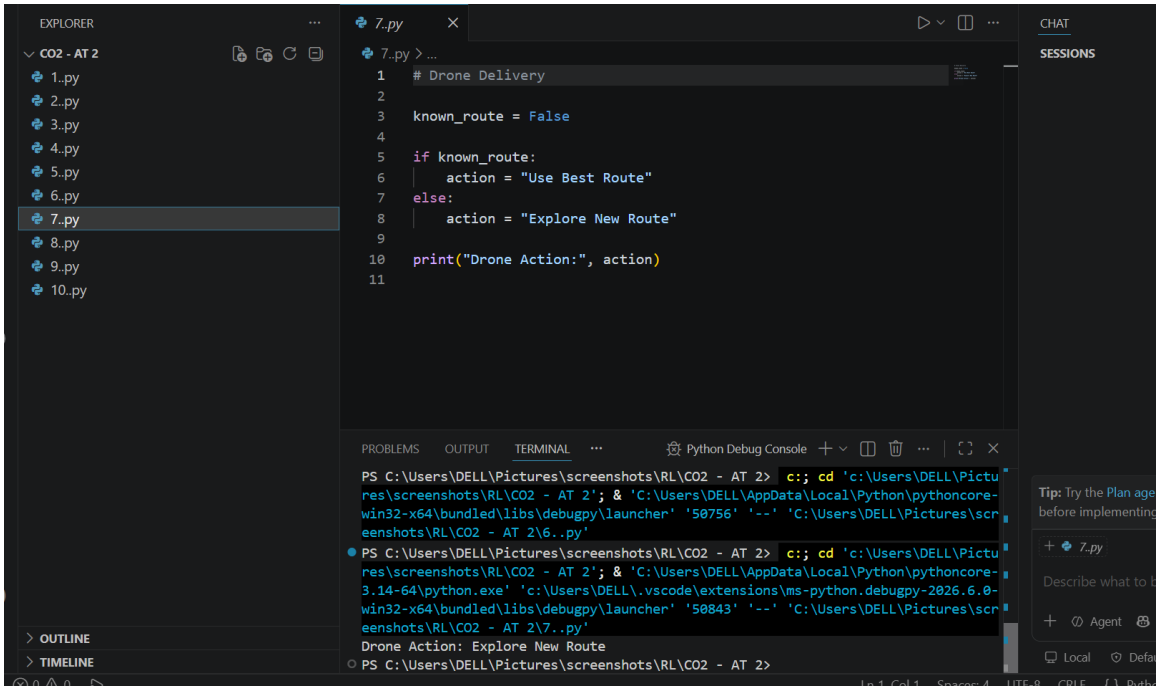
Simple Executable Python Code

```
import random
routes = {"Route A": 8, "Route B": 5, "Route C": 7}
```

```
if random.random() < 0.2:
    route = random.choice(list(routes))
    strategy = "Exploration"
else:
    route = min(routes, key=routes.get)
    strategy = "Exploitation"
print("Strategy:", strategy)
print("Selected Route:", route)
```

Note: This code is intentionally simple and demonstrates the RL state–action–reward/decision idea. A complete RL system learns a policy through repeated interactions.

CODE:



8. Manufacturing Robot Assembly

About the Question

This question is about using RL to improve manufacturing robot assembly operations.

How RL is Used

The robot observes parts, its position and the current task. It chooses actions such as pick, move, place and assemble. Correct fast work receives positive rewards; errors and delays receive penalties.

RL Components

Agent: Manufacturing robot.
Environment: Assembly line, tools and parts.
State: Part position, robot position and current task.
Action: Pick, move, place or assemble.
Reward: Positive for correct efficient assembly; penalty for errors/delays.
Policy: Strategy for choosing assembly actions.

Policy and Value Function

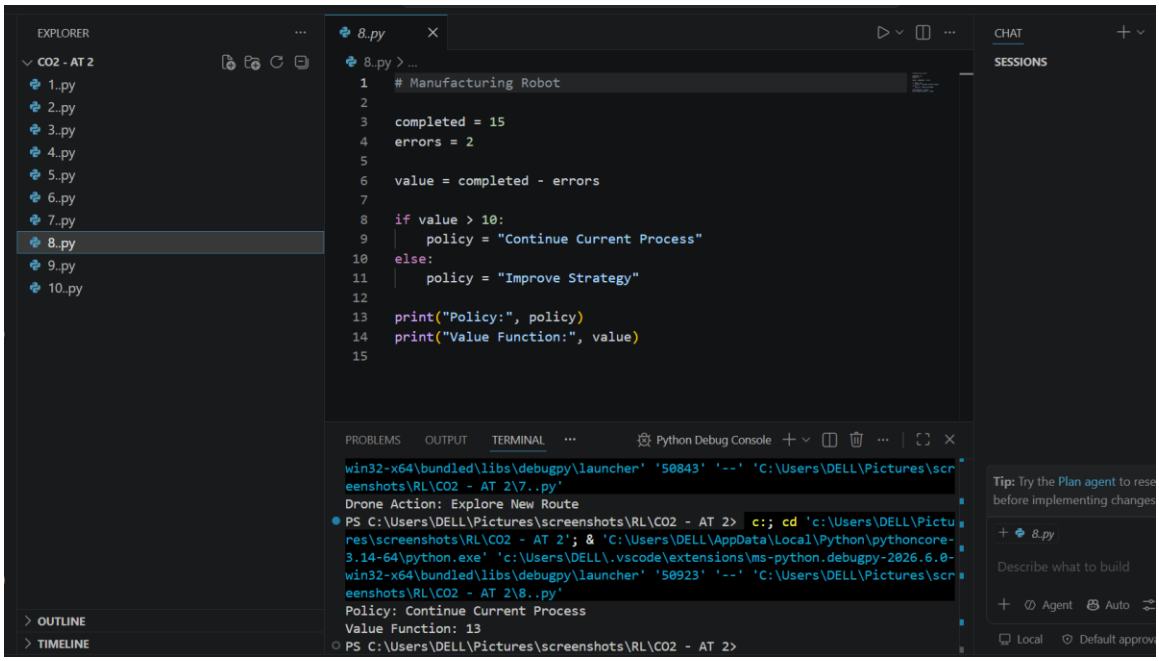
The policy selects an action. The value function estimates expected future reward from a state. Higher-value states guide the robot toward efficient assembly.

Simple Executable Python Code

```
assembly_steps = {"Pick Part": 5, "Place Part": 8, "Assemble": 10}
for action, value in assembly_steps.items():
    print(action, "Value:", value)
best_action = max(assembly_steps, key=assembly_steps.get)
print("Best Action:", best_action)
```

Note: This code is intentionally simple and demonstrates the RL state–action–reward/decision idea. A complete RL system learns a policy through repeated interactions.

CODE:



9. Online Advertising

About the Question
This question is about using RL to select advertisements that improve click-through rate (CTR).

How RL is Used
The system observes user interests and previous interactions, displays an ad and receives feedback from clicks or other useful engagement.

RL Components
Agent: Advertising system.
Environment: Users, website/app and advertisements.
State: User interests, browsing behaviour and previous interactions.
Action: Select an advertisement.
Reward: Positive for useful engagement such as clicks/conversions.
Policy: Strategy for choosing ads.

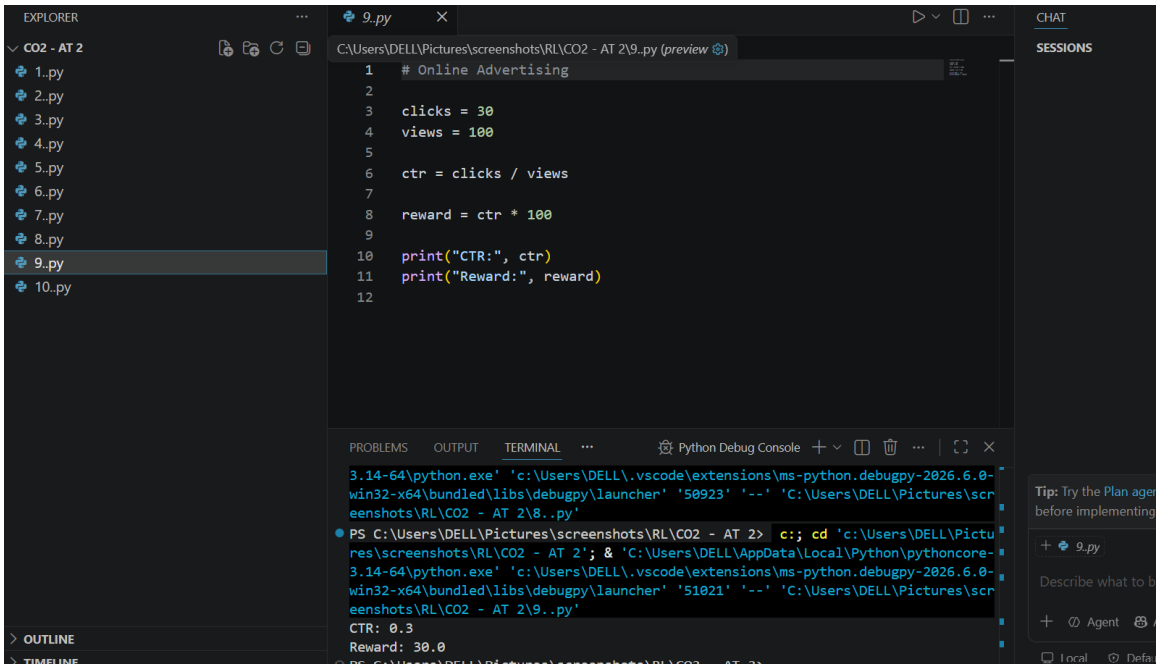
Reward Design Challenges
If reward considers only clicks, the system may learn to show repetitive or clickbait ads. A better reward can also consider conversions, relevance and long-term user experience.

Simple Executable Python Code

```
ads = {"Ad A": 0.05, "Ad B": 0.12, "Ad C": 0.08}
for ad, ctr in ads.items():
    print(ad, "CTR:", ctr)
best_ad = max(ads, key=ads.get)
print("Selected Ad:", best_ad)
print("Reward:", ads[best_ad])
```

Note: This code is intentionally simple and demonstrates the RL state–action–reward/decision idea. A complete RL system learns a policy through repeated interactions.

CODE:



10. Smart Irrigation System

About the Question

This question is about using RL to optimize water usage for crops.

How RL is Used

The agent observes soil moisture, weather, temperature, rainfall and crop needs. It decides when and how much to irrigate. The aim is to maintain healthy crops while avoiding water waste.

RL Components

Agent: Smart irrigation controller.
Environment: Farm, soil, crops, weather and irrigation system.
State: Soil moisture, temperature, rainfall and crop condition.
Action: Supply more water, less water or stop irrigation.
Reward: Positive for suitable soil moisture with low water use; penalty for waste or very dry soil.
Policy: Strategy for deciding irrigation.

Main Idea

Learning from soil and weather conditions helps avoid unnecessary watering and improves water efficiency.

Simple Executable Python Code

```
soil_moisture = 30
if soil_moisture < 40:
    action, water, reward = "Start Irrigation", 20, 10
elif soil_moisture > 70:
    action, water, reward = "Stop Irrigation", 0, 10
else:
    action, water, reward = "Low Irrigation", 5, 8
print("Soil Moisture:", soil_moisture)
print("Action:", action)
print("Water Supplied:", water)
print("Reward:", reward)
```

Note: This code is intentionally simple and demonstrates the RL state–action–reward/decision idea. A complete RL system learns a policy through repeated interactions.

CODE: